

Technical Documentation: Intelligent Parking Compliance System

Dhruv

1 System Overview

The Intelligent Parking Compliance System (IPCS) is a high-performance computer vision solution engineered for autonomous, real-time parking enforcement. By integrating a dual-model neural architecture, the system performs intelligent infrastructure mapping and vehicle occupancy analysis, identifying illegal parking violations without the necessity for manual, site-specific coordinate calibration.

2 System Architecture

The system leverages an integrated two-tier model framework:

1. **Infrastructure Mapping Engine (PKLot-trained YOLO11):** A specialized convolutional neural network trained on the PKLot dataset. This engine performs semantic segmentation and object detection to localize all valid parking bays within a live camera feed, generating a dynamic, high-fidelity spatial "Grid Map."
2. **Vehicle Detection Engine (YOLO11 Medium):** A robust, industry-standard object detection model utilized for the real-time classification and localization of motorized vehicles, including cars, motorcycles, buses, and heavy-duty trucks.

3 Operational Methodology

The IPCS enforces compliance through a systematic, four-stage processing pipeline:

1. **Dynamic Semantic Grid Mapping:** The Infrastructure Mapping Engine performs inference on the input frame to output bounding box coordinates for all legal parking spaces. This dynamic approach eliminates the need for static, manual ROI (Region of Interest) definitions.
2. **Vehicle Object Detection:** The Vehicle Detection Engine performs concurrent inference to identify all vehicular entities within the scene.
3. **Spatial Grounding:** To mitigate errors caused by perspective distortion or vehicle height, the system calculates a "Tire Contact Point"—defined as the bottom-center of the vehicle's detected bounding box. This coordinate serves as the sole ground-truth reference for compliance.
4. **Violation Logic:** The system executes a spatial cross-reference between the calculated Tire Contact Points and the active Grid Map. Vehicles whose contact points lie outside all mapped parking bay boundaries are autonomously classified as illegal, triggering an immediate violation flag.

4 Technical Challenges and Mitigation Strategies

4.1 Mitigating Contextual Hallucinations

Challenge: Vehicles in transit through traffic aisles were frequently misclassified as "parked" due to morphological similarities with parked cars. **Mitigation:** We implemented a strict dual-model validation dependency. A vehicle is only recognized as "parked" if its spatial signature correlates with the infrastructure-defined grid, effectively decoupling vehicle detection from parking status.

4.2 Scalability and Environment Adaptation

Challenge: Traditional systems requiring hardcoded parking zones are non-scalable in urban deployments with varied camera angles. **Mitigation:** Our system utilizes Dynamic Semantic Mapping. The Infrastructure Mapping Engine re-interprets the parking environment for every individual frame, allowing the system to maintain accuracy across diverse camera orientations without requiring recalibration.

4.3 Object Detection Jitter

Challenge: Using central bounding box coordinates induced spatial misalignment for large vehicles (e.g., buses), leading to false positive violations. **Mitigation:** We engineered a contact-point detection algorithm with a 10% vertical offset from the bounding box base, ensuring that the compliance logic remains anchored to the road surface, regardless of vehicle dimensions.

5 Performance Metrics

Validated against the PKLot dataset using a 50-epoch training regimen, the system yields the following performance benchmarks:

- **Mean Average Precision (mAP50-95):** 0.992
- **Mean Average Precision (mAP50):** 0.998
- **Inference Latency:** 3.5ms per frame (~285 FPS)
- **System Throughput:** Optimized for real-time, multi-stream deployment.

6 Implementation Stack

- **Core Framework:** Ultralytics YOLO11
- **Computer Vision Library:** OpenCV 4.x
- **Spatial Processing:** NumPy
- **Analytical Visualization:** Matplotlib / Pandas